

University of British Columbia

ELEC291: Electrical Engineering Design Studio I

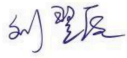
Winter 2025-26

Instructor: Dr. Jesús Calviño-Fraga

Section 201

Project 1: Reflow Oven Controller

Group #: C12

Student name (student number)	Points	Signature
Daniel Gu (96007034)	102	
How Yee Chaw (32416323)	102	
Jacky Zou (32458911)	92	
Yichen Liu (16662876)	102	
Qijun Zhou (40121204)	102	
TOTAL	500	

Date of Submission: February 27, 2026

Table of Contents

I.	Introduction	
	A. Design Objective & Specifications	p.2
	B. Overview of Design Approach	p.3
II.	Investigation	
	A. Idea Generation	p.4
	B. Investigation Design	p.5
	C. Data Collection	p.6
	D. Data Synthesis	p.6
	E. Analysis of Results	p.6
III.	Design	
	A. Use of Process	p.7
	B. Need and Constraint Identification	p.8
	C. Problem Specification	p.8
	D. Solution Generation	p.9
	E. Solution Evaluation	p.10
	F. Detailed Design	p.10
	G. Solution Assessment	p.18
IV.	Live-Long Learning	p.18
V.	Conclusions	p.19
VI.	References	p.20
VII.	Bibliography	p.20
VIII.	Appendices	p.21

I. Introduction

A. Design Objective & Specifications

This project involves the design, construction, programming, and testing of a Reflow Oven Controller using a DE10-Lite microcontroller and a CV-8052 processor. The control system is designed to operate a toaster oven in 5 different stages of the reflow soldering process – ramp to soak, preheat/soak, ramp to peak, reflow, and cooling^[1] – and is implemented by a Finite State Machine (FSM) written in the 8052-style assembly language with the use of a solid-state relay box. We measure the temperature using a thermocouple wire and amplify the voltage difference using an op amp. The system also involves the use of pushbuttons, an LCD displaying important parameters, a buzzer for sound feedback, and a live parameter-selection and temperature-plotting program generated using Python, allowing our product to be interactive and more convenient to use. Furthermore, a vital safety feature is added to the system, which turns the oven off if the temperature hasn't reached at least 50°C in the first 60 seconds of operation, preventing the oven from overheating and causing potential damage. Below are the diagrams of the hardware and software.

B. Overview of Design Approach

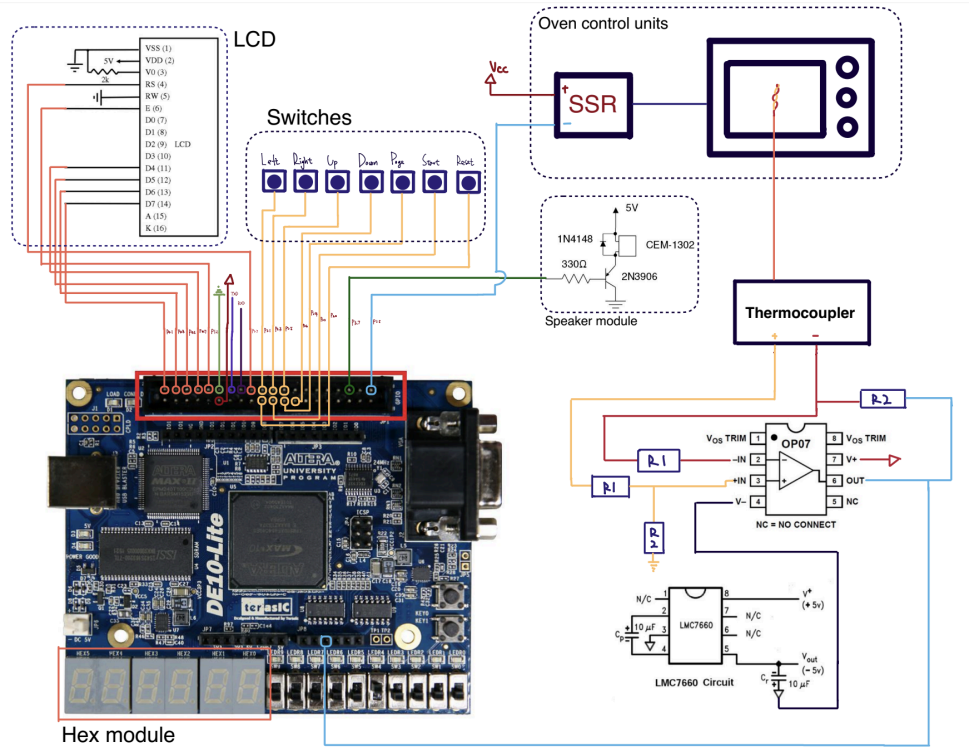


Figure 1: Hardware Design System Block Diagram (refer to Appendix E)

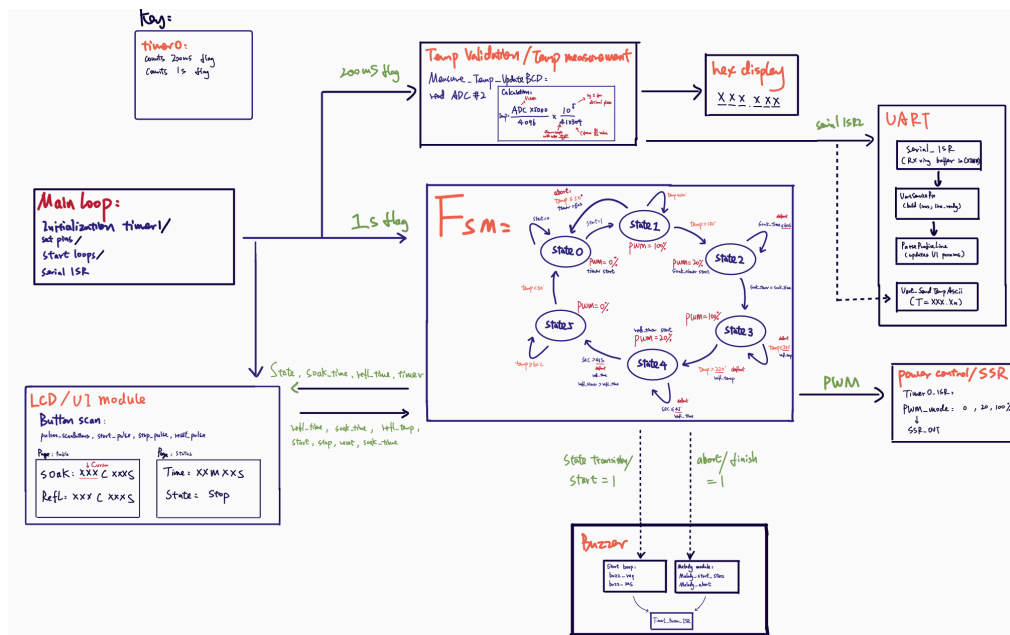


Figure 2: Software Design System Block Diagram (refer to Appendix F)

II. Investigation

A. Idea Generation

We selected the solid-state relay (SSR) box, a device that “switches on or off when an external voltage (AC or DC) is applied across its control terminals,”^[2] to control the toaster oven’s power output. To allow our DE10-Lite microcontroller to communicate with the SSR, we connected the microcontroller’s 5V pin to the SSR’s positive input terminal and the microcontroller’s pin P1.5 to the negative input terminal. Additionally, a 2N7000 N-MOSFET is added to make this communication more reliable, with its gate connected to P1.5 and its source sharing the same GND as the DE10-Lite.

Furthermore, our team believes that the K-type thermocouple is the best choice for measuring the live temperature inside the oven, as it is known to be “inexpensive, accurate, reliable, and has a wide temperature range”^[3] suitable for 25-240°C. The thermocouple measures the temperature difference between the oven (hot junction) and room temperature (the cold junctions), while the LM335 measures room temperature for cold-junction compensation.

Because the thermocouple’s output voltage is negligibly small, we amplified the voltage signal using an operational amplifier (op-amp) and four resistors before feeding it into the DE10-Lite ADC. This voltage signal is then converted into the measured oven temperature in °C via an 8052-style assembly software. Nevertheless, considering factors such as the op-amp’s offsets, the mismatch between the resistors’ actual and colour-coded resistance, and potential errors in the software’s voltage-temperature conversion, inaccuracies may be present in our system’s temperature measurement, indicating that temperature validation is needed. We thus hypothesize that by using the K-type thermocouple, the LM335, the OP07 with amplifying

resistors, as well as the formula $T_{oven} = T_{cold} \text{ (room temperature)} + \Delta T_{K-couple}$ in our assembly program, our system can accurately monitor and compute the oven's internal temperature.

B. Design Investigation

To test the accuracy of our temperature measuring system, we use a Tektronix DM4020 multimeter to measure the thermocouple's output voltage, as well as a Python program to convert this voltage into the reference oven temperature in degrees Celsius using this formula:

$$T_{oven,ref} = T_{cold} + \Delta T = T_{cold} + \frac{V_{K-couple}}{41\mu V/^{\circ}C}, \text{ where } T_{cold} = 22^{\circ}C$$

On the other hand, the measured value for oven temperature is computed using voltage signals received by the DE10-Lite and an 8052 assembly software running the following formulae:

$$\Delta T = \frac{V_{K-couple}}{41\mu V/^{\circ}C} = \frac{V_{out}/300}{41\mu V/^{\circ}C} \quad (V_{out}: \text{voltage signal received by DE10-Lite ADC})$$

$$T_{cold} = \frac{V_{LM335}}{0.01V/^{\circ}C} - 273 \quad (V_{LM335}: \text{voltage out of LM335})$$

$$T_{oven,measured} = T_{cold} + \Delta T$$

The difference between these two temperature measurements is then continuously computed by the Python software and monitored in real time, ensuring that it remains within an acceptable range, and validating the accuracy of the measurement system.

As shown in Figure 1, we have selected the OP07 as our op-amp due to its minimal offsets, which help reduce measurement error. We have also chosen the resistor values $R1 = 1k\Omega$ and $R2 = 300k\Omega$. With this ratio, the LM355's voltage output is, theoretically, amplified by a factor of 300 ($300k/1k = 300$) before reaching the DE10-Lite board. This gain ensures that the op amp is still saturated even at the highest temperature to measure, $240^{\circ}C$. Furthermore, to calibrate the op-amp's input voltage to $\pm 5V$, an LMC7660 voltage converter is used instead of a voltage divider, and its detailed connection to the op-amp is sketched in Figure 1.

C. Data Collection

We designed a Python program that continuously prints $T_{oven,ref}$, $T_{oven,measured}$, and the temperature error $|T_{oven,ref} - T_{oven,measured}|$ to the terminal. These values are printed every second with a line break separating each set of data. Additionally, to efficiently test our system over a wide range of temperatures, a soldering iron was used to heat the thermocouple's hot junction to a high temperature (approximately 250°C), and then allow it to cool back to room temperature. This enabled us to observe how the temperature error varied across different temperature ranges and assess the system's performance under both heating and cooling conditions.

D. Data Synthesis

The success of our temperature validation is determined by whether there are frequent or sustained temperature errors above 3°C in the test results. Temperature validation may also fail if the error value starts to jump above 3°C when the hot junction undergoes significant heating or cooling, as this suggests instability in the measurement system.

The final numerical data for temperature validation has been converted to an Excel table (Appendix C). The data clearly shows that within a 220-second testing time and a temperature range of 53.5~245.5°C, the temperature error never exceeds 3°C. This indicates that despite the temperature errors being large at first, our system has become more reliable and accurate after rounds of debugging and hardware adjustments.

E. Analysis of Results

At the early stage of temperature validation, our group observed that the temperature error between the thermocouple measurement and the multimeter reference was almost always

above 3°C. To identify the root cause, we systematically analyzed both software and hardware factors and synthesized data from multiple sources.

First, we compared the expected amplifier gain, which is based on colour-coded resistor values, with the actual gain calculated from measured resistances. We used the multimeter to measure the true resistances of all four resistors, and found noticeable deviations from their colour-coded values. Based on this observation, our group recalculated the op amp gain and updated the value in our assembly software from 300 to 304. This modification reduced the frequency of temperature error exceeding 3°C, suggesting that resistor tolerance contributed partially to the problem.

However, occasional temperature spikes above 3°C still occurred, leading our group to synthesize this result with additional observations about the hardware condition. Notably, the OP07 used in our circuit had been previously used in several labs. Considering the possibility of component degradation, we replaced it with a new OP07, and the temperature error finally started to stabilize within a range of 0~2.88°C.

By integrating measured electrical data (resistor values), software recalibration (gain adjustment), and hardware substitution (op-amp replacement), we concluded that both resistor tolerance and the op-amp's condition contributed to the error. The final design achieved reliable and consistent temperature validation results, as indicated in Appendix C.

III. Design

A. Use of Process

Throughout the process of creating the reflow oven, our group used the general engineering design process by breaking down the entire project into multiple blocks and then

making tests, analyzing, and iterating on each block in various different methods. We split the entire project into these blocks: 1. Temperature Sensing, 2. Building FSM, 3. LCD + buttons + speaker control, 4. GUI interaction via UART. One example of the iteration would be the temperature validation part of the reflow oven. We faced many challenges regarding the hardware, and had to debug many of our wires and components to make sure they worked. Also, we tested many different approaches, such as calibrating the op amp and the LM335, to create the easiest yet still very accurate temperature measurement.

B. Need and Constraint Identification

The system must reliably control oven power through the SSR box and accurately measure temperature over the operating range. The controller must also remain safe and user-controllable during operation, including immediate shutdown behaviour when the run is stopped and aborting condition when the temperature rise indicates the sensor is not placed properly.

Several constraints influenced our design decisions. On the hardware side, we needed to manage SSR switching, temperature sampling and calculations, and timing using limited embedded resources. On the software side, the system had to remain responsive at all times—long blocking delays could slow down a shutdown, miss button inputs, or disrupt serial communication. This constraint became even more critical after adding our extra features, since they increase CPU workload and can reveal timing problems unless the control loop is organized in a non-blocking, well-scheduled manner.

C. Problem Specification

From these needs and constraints, we separated the project into two categories: basic features that must be corrected first, and extra features added only after the basics are stable. The

basic feature set includes reliable SSR-based power control, correct temperature measurement and computation, and successful temperature validation within $\pm 3^\circ\text{C}$ across 25240°C ; these formed the foundation of a safe reflow controller. On top of that, we implemented three extra features to improve usability and demonstration quality: UART temperature streaming for a 1 Hz PC strip chart, a button-driven interface with two LCD pages, and music/alerts through a buzzer. The overall specification required that these extra features must not degrade the basic functions—specifically, SSR switching and validated temperature accuracy had to remain consistent and reliable even while the UI, UART, and audio were active.

D. Solution Generation

We explored and tested multiple approaches before settling on an integrated solution that protects the basic features. During early testing, we tried LM335 and LM4040-based setups as part of our investigation, but since the op amp output already met the validation accuracy requirement, we chose not to introduce additional calibration/offset, and did not include those components in the final design. For any analog scaling that depended on resistor ratios, we used measured resistor values rather than nominal ratings, improving repeatability without adding extra effort for the user.

For the extra features, we implemented a UART output format suitable for a 1 Hz strip chart and designed it to remain stable through state transitions. We also added a button-driven UI with two LCD pages so the controller could present both configuration and run-time status cleanly. Finally, we implemented buzzer music and alert patterns to provide clear audible feedback without requiring the operator to continuously monitor the display. To keep the basic control loop responsive and prevent UART timeouts or delayed input handling, we structured timing and updates in a non-blocking style rather than relying on long busy-wait delays.

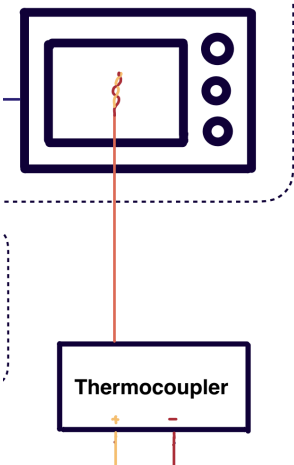
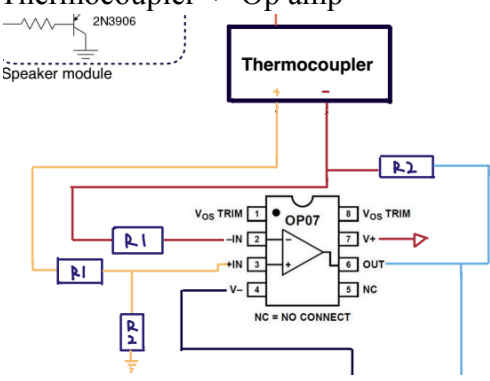
E. Solution Evaluation

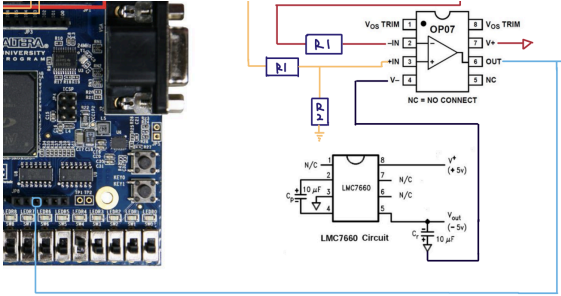
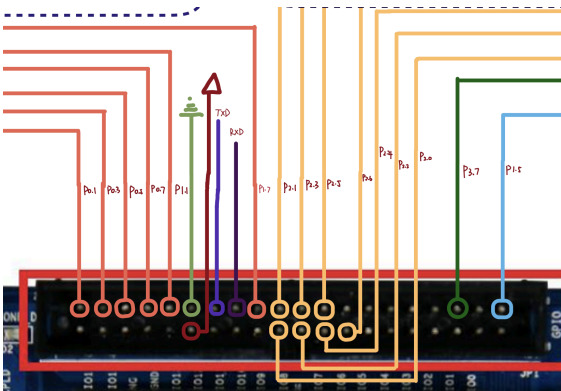
The final solution worked well because it prioritized correct core functionality while adding extra features that improve usability and clarity of demonstrations. By keeping SSR control and validated temperature measurement as the foundation, the controller satisfied the most critical project requirements and reduced risk during integration. Avoiding unnecessary recalibration (after confirming sufficient accuracy) also reduced complexity and prevented new sources of systematic error.

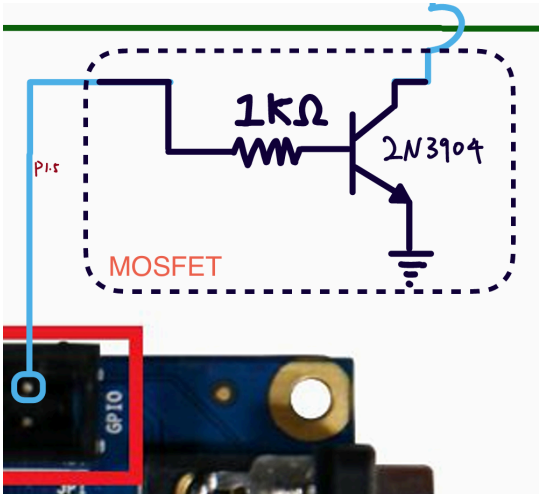
The extra features enhanced the system without compromising the core functions. (UART enabled clearer monitoring of reflow behaviour through a PC strip chart, while the two-page LCD interface and buttons improved usability by separating configuration from run-time information. The buzzer music and alerts added immediate feedback for state changes and end conditions, which is especially useful when the user is not continuously watching the screen. Most importantly, the non-blocking structure ensured these extra tasks did not interfere with responsiveness or safety-related behaviour, preserving immediate shutdown capability and stable control throughout the run.

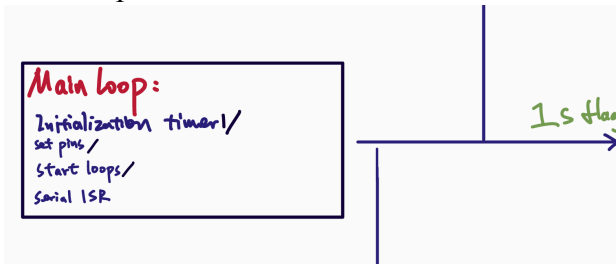
F. Detailed Design

Hardware Block	Wiring	Description
Oven -> Thermocoupler	We twist one end of the thermocoupling wire to make a hot junction, and leave the other side just as it is.	The reason we use thermocoupling wire for this is that it can withstand much higher heat than chips like the LM335, and the absolute measure of the wire is quite accurate. As well, it is relatively

		simple to use, as we just measure the voltage of the thermocouple and calculate the temperature using the ratio 41 microvolts per degree Celsius.
Thermocoupler -> Op amp 	Put thermocoupling into the oven. Attach the thermocouple to the op amp. Attach resistors R1 and R2 with ratio of $R2/R1 = 300$. Connect the LM660 pin configuration and pin 8 to the -5V of the op amp.	Because the voltage measurement from the thermocouple is quite small (microvolts), we use an op amp to amplify by a ratio of $R2/R1$ so that the voltage that gets sent to the microcontroller is larger. To do this, we chose R2 and R1 so that when the oven hits the max temperature at 240 degrees, the op amp is still saturated. The theoretical value of $R2/R1$ came out to be 300.
Op amp -> Microcontroller + LMC7660 voltage converter	To power the op amp, we connect the VCC and -VCC to the LMC7660.	The purpose of the LMC7660 is to produce the -VCC value of -5V instead of using a voltage divider

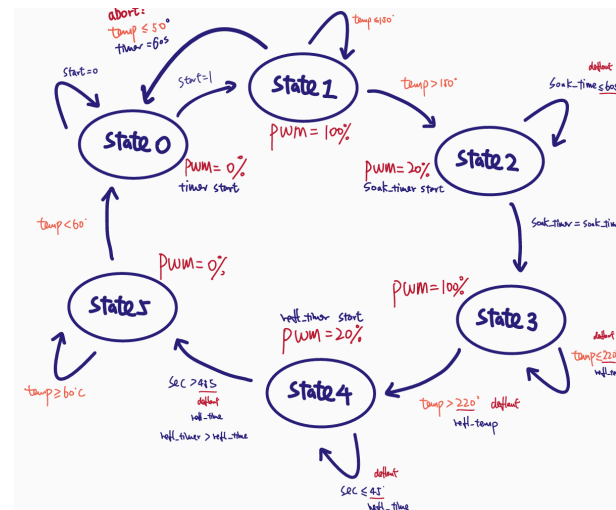
	Then connect the output to the microcontroller's ADC channel #2.	with 10V, as we would need an external power supply for that. We read the output of the op amp voltage into ADC channel 2, which the software will use to calculate temperature.
Microcontroller -> LCD, BO230XS USB adapter, buttons, SSR box, speaker.  Left to Right: LCD pins, VCC + GRD, RXD + TXD for USB adapter, Button pins, speaker pin, SSR box pin.	For the pins, we connected them so that each component's pins were adjacent to each other. We connected 4 pins to D4-D7 for 4bit LCD, and 2 for RS and enable. Then, we connected the RXD and TXD of the microcontroller to the TXD and RXD of the USB adapter. For buttons, we had 7, which we assigned for our respective lcd interaction. Lastly, we connected the negative	One issue we noticed was that some pins could not be used. When we tried running the SSR box the first time, it did not work, but when we changed the pin, it worked. So, the arrangement of the pins came from testing the pins one by one and coming up with ways to make the wiring tidy. For the buttons, we chose them to match our LCD interaction, which will be explained in the software block. Lastly, for the speaker, we chose the configuration that was the loudest to ensure the user hears and gets annoyed by the alarm.

	side of the SSR box and the speaker to the pins on the far left.	
	For the SSR box inputs, we connect pin1.5 on the DE10-lite to the base of the 2N3904(BJT), and connect that into the negative port of the SSR box.	Compared to directly connecting the pin output to the SSR box, we found that using a BJT is much more reliable, and well matched the function of delivering 0, 20 and 100 percent of power.

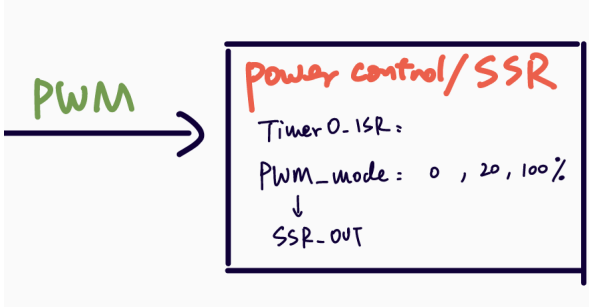
Software Block	Description <i>**refer to Appendix A for the source code</i>
Main loop 	Our main program begins in main: with one-time initialization, then enters the infinite MAIN_LOOP. It configures GPIO for the LCD, oven output (SSR), buzzer, and button inputs, and clears key outputs to ensure the startup state (line 2185). It then initializes the LCD and UART and clears the UART receive/parse state to prevent stale serial data from being processed (line 2206). The program also initializes Timer0 in periodic mode and loads its reload values before enabling the interrupt (line 2172). The Timer0

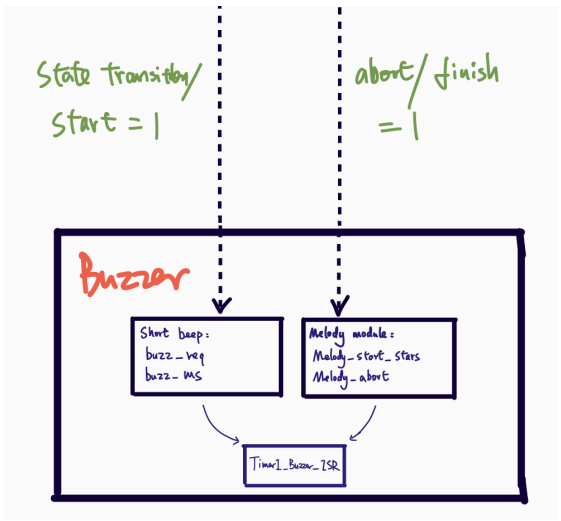
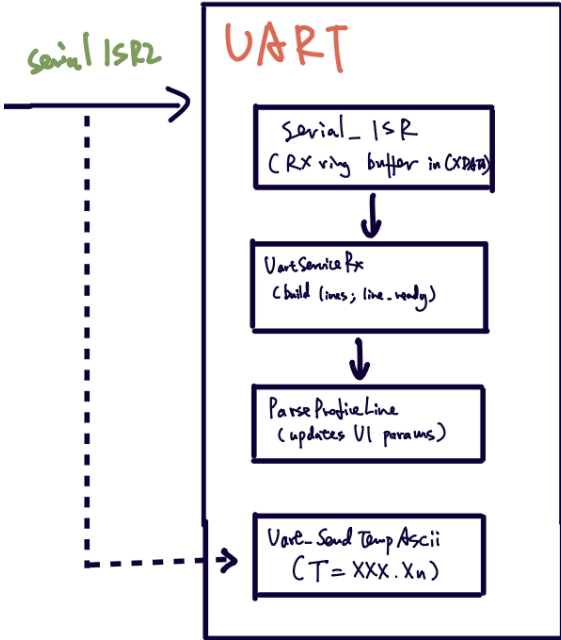
ISR maintains software timing counters and generates two timing flags: a 200 ms flag for temperature reading, button polling and display updates, and a 1-second flag used to advance the FSM and timed reflow stages (line 2105). By deriving both timing references from Timer0, the system achieves consistent real-time behaviour without blocking delays in the main loop. Next, the program initializes core UI/control variables (button state, page/cursor selection, run flags) and loads the default reflow profile parameters (soak/reflow temperatures and times, aka SRTT) (line 2221). After initialization, execution remains in MAIN_LOOP, which repeatedly services UART input, handles button/UI actions based on the 200 ms timing flag, advances the FSM when the 1-second flag is set, updates buzzer stages, and renders the appropriate LCD page.

FSM



The Finite State Machine (FSM) block controls the sequencing of the reflow oven through its operating stages. The FSM is executed once per second from the main loop via lcall FSM_Step_1Hz (line 2395), which is gated by a one-second timing flag check (lines 238). The core FSM routine begins at FSM_Step_1Hz (line 1978), where user inputs such as START, STOP, and RESET are first evaluated

	(line 1979) to allow immediate control overrides. If the system is running, the routine updates state timing counters (line 2012) and branches to the appropriate state handler using a jump structure (line 2015). Individual state handlers are implemented as ST0 through ST5 (line 2021). ST1 handles preheat ramp-up and includes a timeout safety check (line 2031). ST2 maintains the soak temperature for the programmed duration (lines 2065). ST3 ramps to reflow temperature (lines 2078), and ST4 maintains reflow for the specified time (lines 2093). ST5 disables heating and waits for cooldown before returning to idle (lines 2106). Heater output decisions are made inside each state handler, and the display is updated within these same blocks to ensure the LCD reflects the current stage.
Power 	For power, we have three modes of power operations, which are 0,20,100 percent of power. The power module receives the state flags and does the corresponding timer_0_ISR→SSR_OUT. For 20 percent power, we did it as in every 1 sec, 0.2 sec for PWM = 1, 0.8 sec = PWN = 0.
Buzzer	For the buzzer, we made a customized tone (little stars) and a short beep for the state transition. The buzzer receives the

	state transition/start from push button as buzz_req and buzz_ms (line 194); this will trigger the timer1_buzzer_ISR short beep (line 856). When the buzzer module receives an abort or finish signal = melody_start_stars or melody_abort, it will trigger the melody timer_Buzzer_ISR and start the melody.
UART 	The UART block manages serial communication between the CV8052 and a PC for SRTT and temperature reading. The UART is initialized in InitSerialPort (line 299), where Timer2 is configured as the baud-rate generator of 115200, and the serial port is in 8-bit mode with reception enabled. Incoming data is handled using Serial_ISR (line 316), which captures received bytes and stores them in a buffer so that communication doesn't block the main control system. In the main loop, the routine UartServiceRx (line 373) assembles newline-terminated commands and passes complete lines to ParseProfileLine (line 2314) for profile updates. For transmission, acknowledgement routines such as Uart_SendOK (line 639), Uart_SendERR (line 648), and Uart_SendTempAscii (line 673) send formatted ASCII responses using a simple polled transmit routine (line 659).

Temp validation

Temp Validation / Temp measurement

Measure_Temp - Update BCD =
read ADC #2

Calculations:

$$Temp = \frac{ADC \times 5000}{4096} \times \frac{10^5}{41304}$$

Annotations:

- V_{ch2} (pointing to ADC)
- by 5 div decimal places (pointing to 10^5)
- Resistor ratio with ratio (pointing to 41304)
- known $\frac{R1}{R2}$ value (pointing to 41304)

The amplified thermocouple voltage is read from ADC channel #2. This value is then converted into temperature with the formula on the left. It is first converted into Vchannel by multiplying by VCC/ADC_max(which is $2^{12} = 4096$) (line 1013). Then, we convert the thermocouple ratio of 41 microvolts/°C and instead use 10^5 instead of 6 to incorporate decimal points onto our display. Then, we divide by 304, which is the measured resistor ratio R2/R1. We found that by using a measured ratio, our results became more accurate. Lastly, we add the constant 22 degrees, as we decided it was simpler to not incorporate the LM335 + 4040.

LCD/ UI module

LCD / UI module

Button scan:
pulses_scanButtons, start_pulse, stop_pulse, reset_pulse

Page: Profile

SOak: xxxC xxxS
RefL: xxxC xxxS

Page: Status

Time: xxM xxS
State: Stop

The LCD block handles page navigation, cursor movement, and parameter editing using the pushbuttons. The LCD is initialized at startup (line 2206), and button inputs are processed in MAIN_LOOP using the 200 ms timing flag for stable polling (line 2385). A page state variable determines whether the Profile, Status, or Run screen is displayed (line 2445). Within the Profile page, a cursor index tracks which parameter is selected, and the UP/DOWN buttons adjust the highlighted value (line 2460). When changes occur, the corresponding LCD row is refreshed to reflect updated values (line 2500). During operation, the display instead shows the

	current FSM state and countdown timer, updated using the 1-second flag to maintain real-time accuracy.
--	--

G. Solution Assessment

The “Temp Validation Graph” in the Appendices presents the temperature data recorded during the “preheat,” “ramp-to-peak,” and “reflow” stages of a test run conducted during system validation. The results match the expected reflow profile, with the temperature remaining mostly stable during the “preheat” and “reflow” states and increasing steadily during the “ramp-to-peak” stage. As well, when we were deciding on what hardware we used, we first tried finding a cold temperature with LM335 and LM4040 but noticed that the error was still around the same without using these two components, so we stuck with just using 22 degrees constant (Refer to Appendix G for LM335/4040 and Appendix C for 22 constant). As well, we calibrated the op amp, believing that the error would be better, but again, no real results showed improved accuracy. The only fix on the hardware that made a difference was to use measured resistance values instead of the colour code, which only helped accuracy slightly. On the software side, one of the largest solutions we implemented was to make sure nothing was on “busy wait,” and most things were triggered through flags and counters instead. We noticed a noticeable difference in the LCD’s response when changing from blocking to non-blocking.

IV. Live-Long Learning

This project showed us that our main learning gap was in building a large embedded system that stays responsive while executing several tasks at once. We were comfortable with basic I/O and small assembly programs, but we had not previously worked on a design that

needed reliable SSR power control, continuous temperature sampling/computation, and real-time user feedback to all run together without blocking delays. We had to teach ourselves how to structure timing and program flow so that features like Stop, UART updates, and LCD/button handling would not interfere with the core control loop. We also learned more practical skills in validating measurements, including paying attention to real component values and how tolerances affect temperature calculations, which helped us meet the required accuracy during testing.

V. Conclusions

In this project, our group successfully implemented a reflow oven control system run by the DE10-Lite and CV-8052 microcontrollers. This comprehensive system integrates temperature measurement, FSM control, SSR power regulation, as well as interactive features including the LCD user interface, UART, and sound feedback. The system is able to operate stably according to a pre-set reflow profile, and temperature validation results demonstrate that the error can be effectively controlled within $\pm 3^{\circ}\text{C}$, satisfying the project's accuracy and safety requirements.

On the hardware side, we used a K-type thermocouple together with an OP07 amplifier circuit for real-time temperature measurement. Through multiple rounds of testing, we discovered that resistor tolerances and op-amp conditions significantly affected measurement accuracy. By measuring actual resistor values, recalculating the gain in software, and replacing aging components, we were able to stabilize the system and achieve reliable results. These debugging stages required significant time and effort, and careful comparisons between theoretical calculations and experimental data.

Regarding software, we adopted a non-blocking architecture based on timer interrupts. We faced several integration challenges during the design process, such as UART and buzzer

conflicts, timing inconsistencies, and responsiveness issues when multiple modules ran simultaneously. These problems required us to debug the program multiple times, ensuring that FSM control, temperature computation, and user interaction could operate concurrently.

Overall, this project required a significant time commitment outside scheduled lab hours. As a group, we spent about 55 hours throughout the process, from debugging hardware configurations to validating temperature accuracy. While the process was challenging, it strengthened our ability to systematically troubleshoot complex embedded systems and deepened our understanding of engineering design processes, error analysis, and system integration. These experiences laid a solid foundation for the future development of even more complex embedded systems.

VI. References

- [1] “Electronics Soldering Process.” *Wikipedia.org*, Wikimedia Foundation, Inc., 18 Jan. 2005, en.wikipedia.org/wiki/Reflow_soldering
- [2] “Solid-state relay.” *Wikipedia.org*, Wikimedia Foundation, Inc., 17 Nov. 2006, en.wikipedia.org/wiki/Solid-state_relay
- [3] “Type K Thermocouple.” *Thermocoupleinfo.com*, 2026, www.thermocoupleinfo.com/type-k-thermocouple.htm

VII. Bibliography

- Harris, S., and Harris, D., “Digital Design and Computer Architecture, RISC-V Edition,” Morgan Kaufmann, 2022.
- Terasic Inc., “DE10-Lite User Manual,” September 7, 2016.

VIII. Appendices

https://drive.google.com/drive/folders/1tSyhEfmelJ0EcK1ZTfsrpP18ZTjlbwg2?usp=drive_link

The above link gives access to all the appendices (Appendix A - G)